



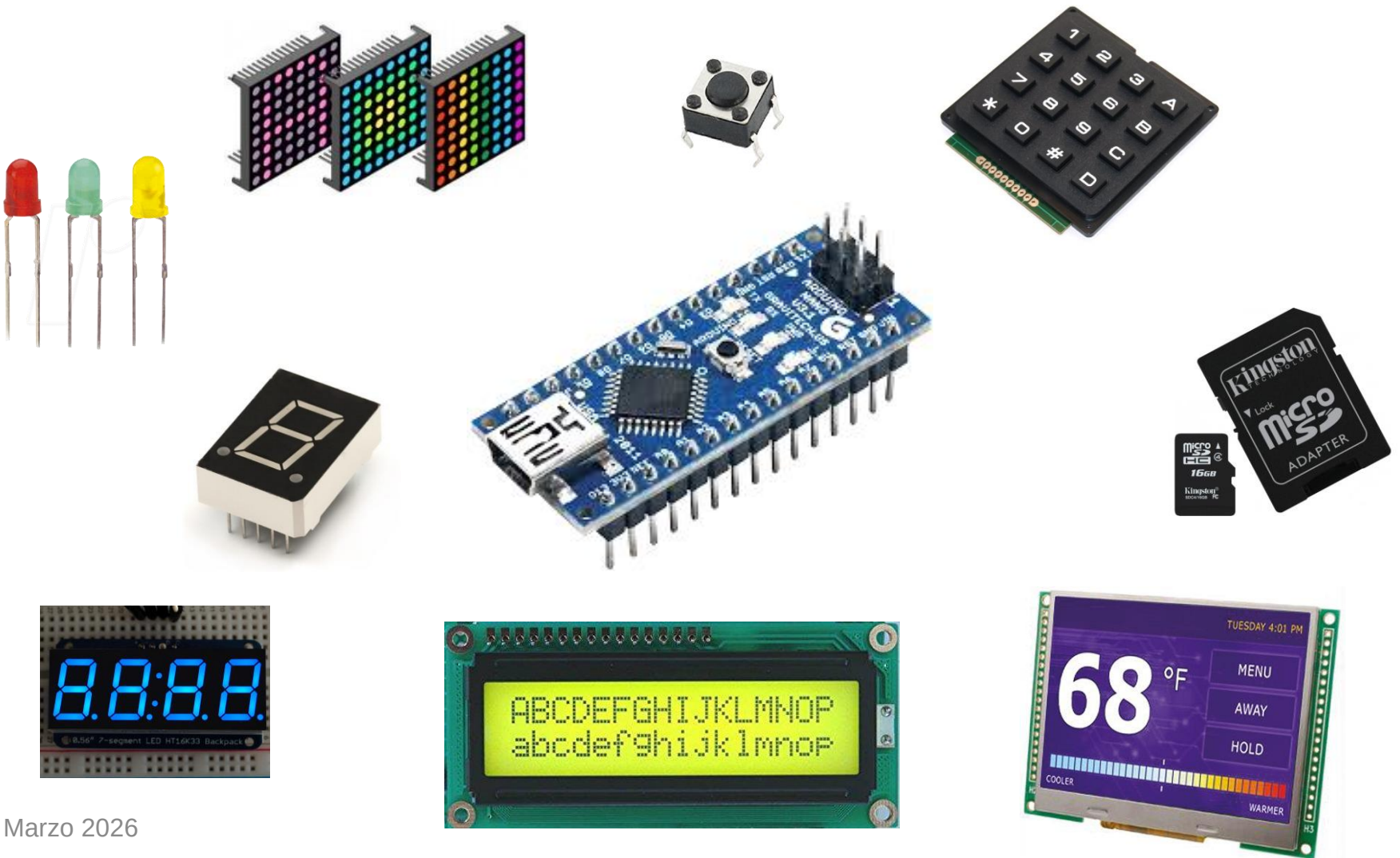
CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

Programación de puertos de
Entrada/Salida

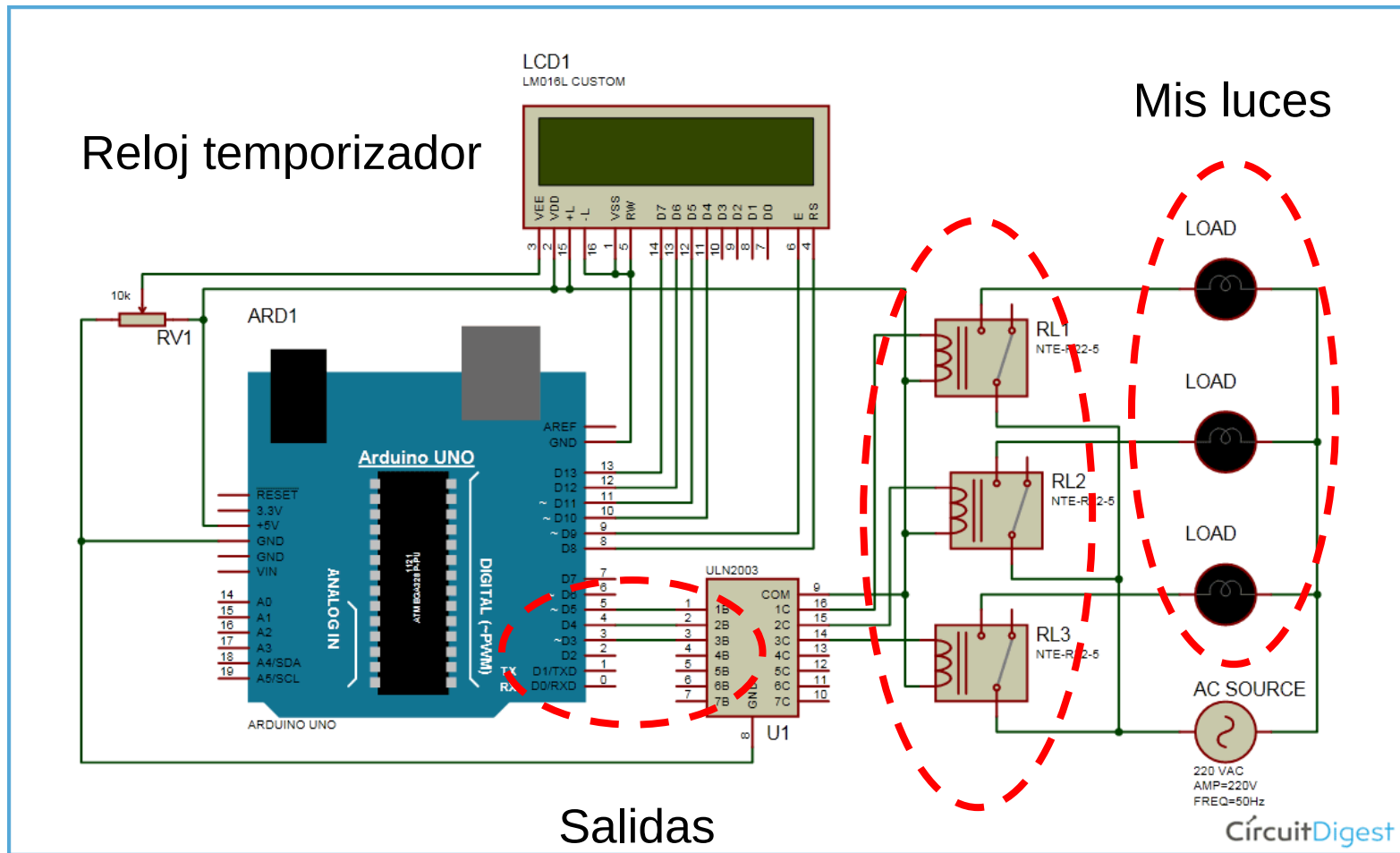
TP1

¿Puertos de Entrada-Salida?

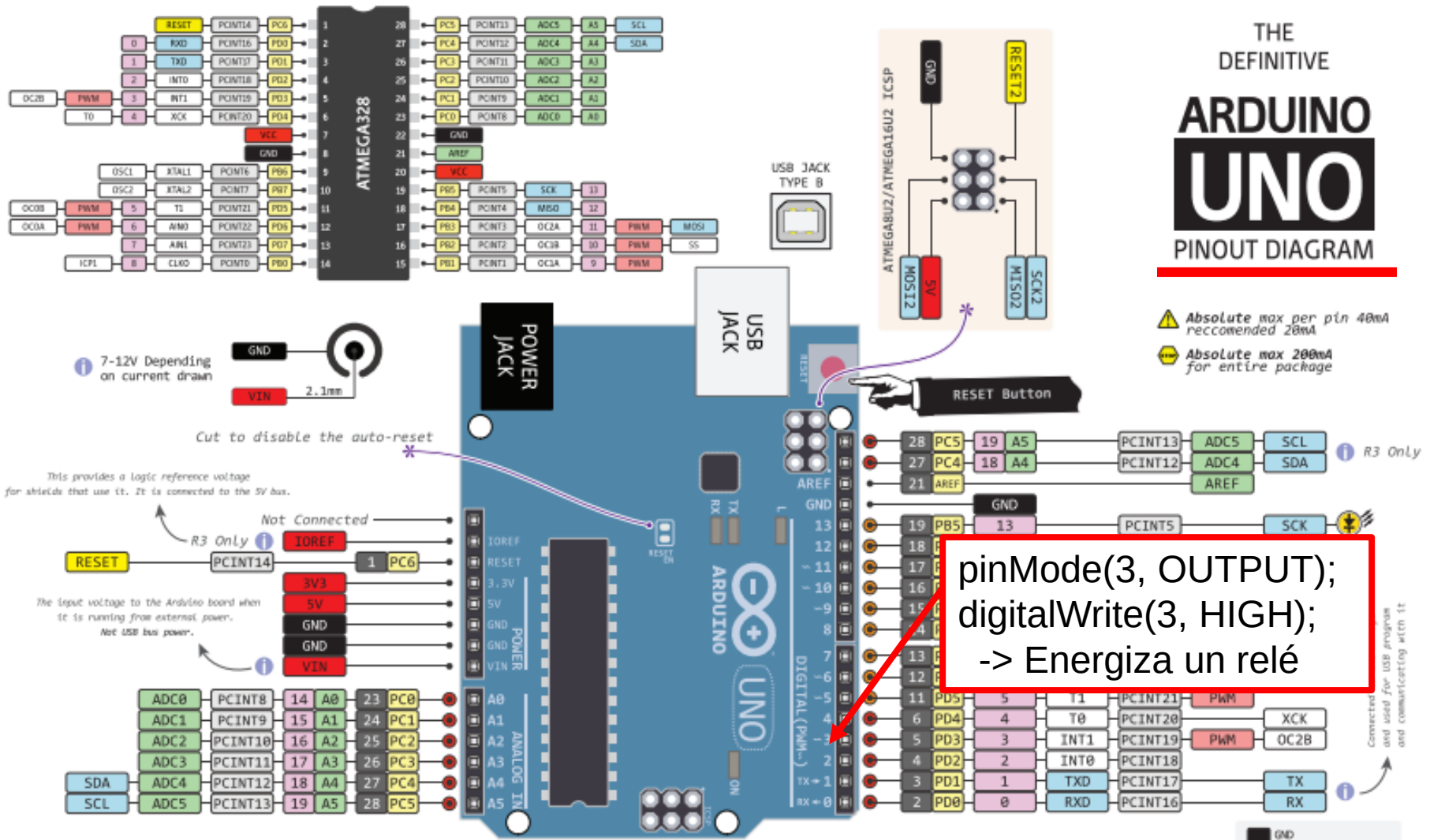


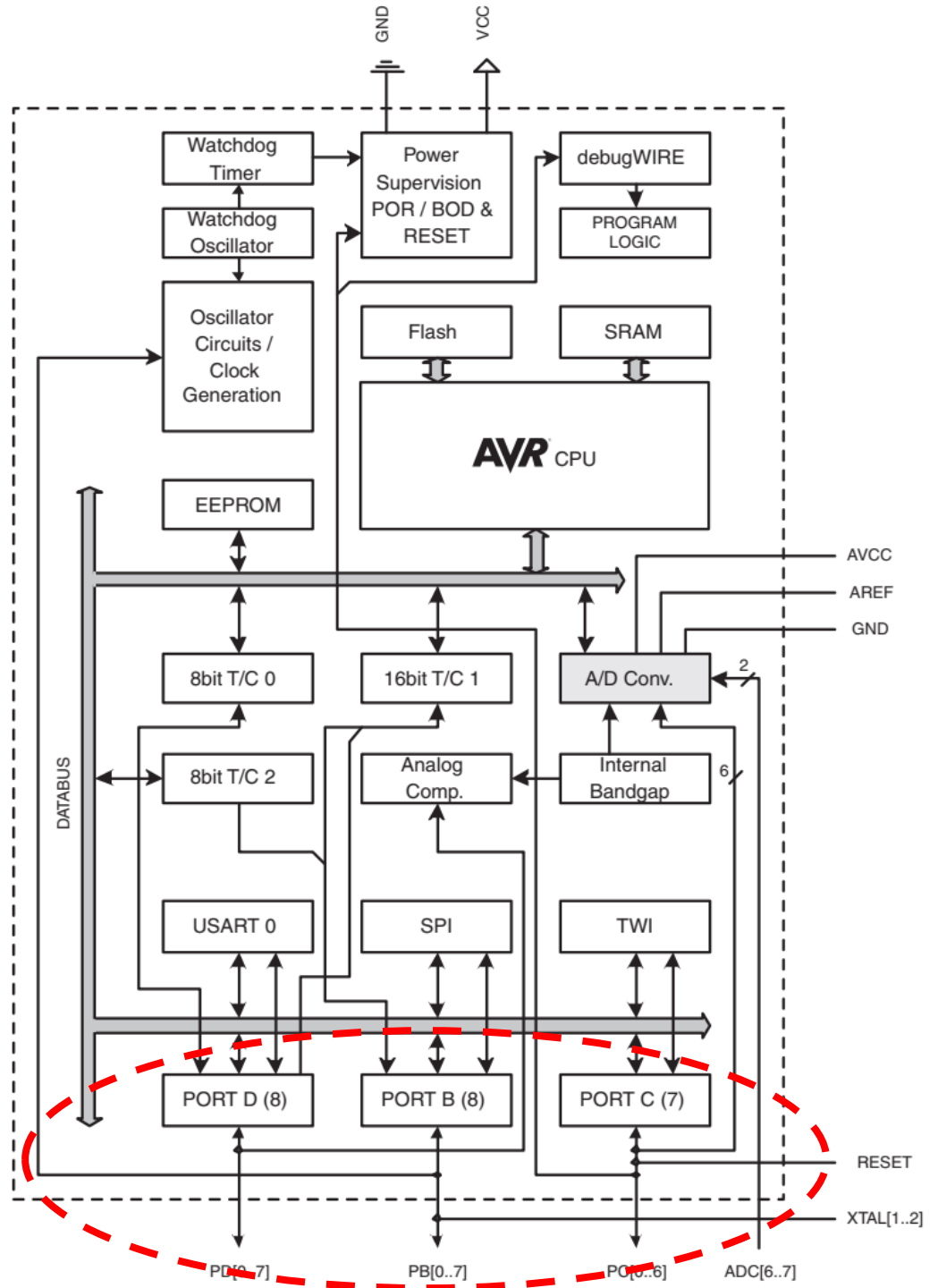
Un ejemplo motivador

Domótica Fácil



Arduino





ATMEGA - AVR

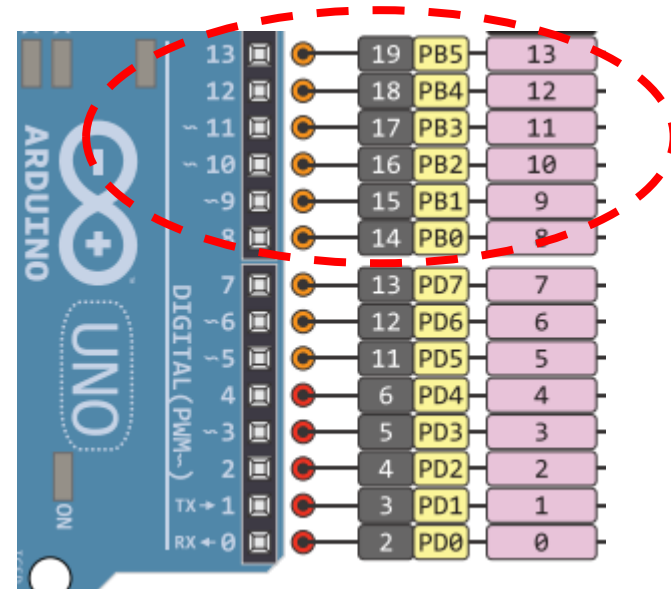
¿Donde están los terminales I/O del puerto B?

Pinout ATmega48PA/88PA/168PA/328P

Pin diagram of the ATmega328P microcontroller. The diagram shows the pin numbers, functions, and connections for each pin. Pins 8 and 21 are connected by a dashed red line, and pins 9, 10, 14, and 15 are connected by a dashed red line.

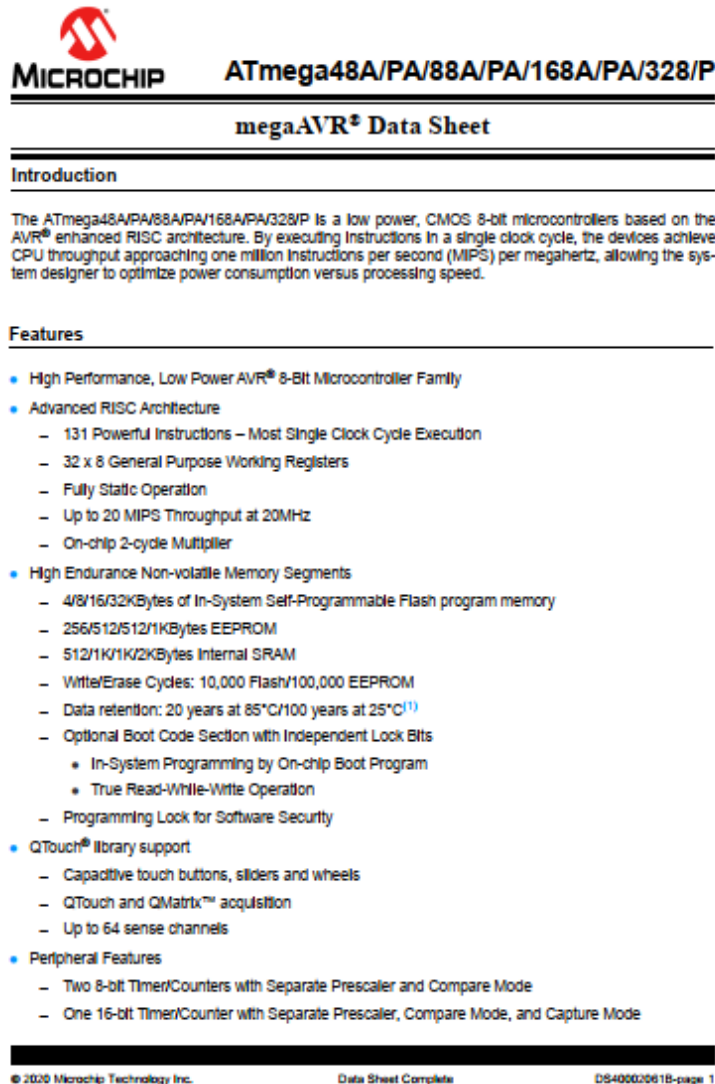
Pin	Function
1	(PCINT14/RESET) PC6
2	(PCINT16/RXD) PD0
3	(PCINT17/TXD) PD1
4	(PCINT18/INT0) PD2
5	(PCINT19/OC2B/INT1) PD3
6	(PCINT20/XCK/T0) PD4
7	VCC
8	GND
9	(PCINT6/XTAL1/TOSC1) PB6
10	(PCINT7/XTAL2/TOSC2) PB7
11	(PCINT21/OC0B/T1) PD5
12	(PCINT22/OC0A/AIN0) PD6
13	(PCINT23/AIN1) PD7
14	(PCINT0/CLKO/ICF1) PB0
15	PB1 (OC1A/PCINT1)
16	PB2 (SS/OC1B/PCINT2)
17	PB3 (MOSI/OC2A/PCINT3)
18	PB4 (MISO/PCINT4)
19	PB5 (SCK/PCINT5)
20	AVCC
21	AREF
22	GND
23	PC0 (ADC0/PCINT8)
24	PC1 (ADC1/PCINT9)
25	PC2 (ADC2/PCINT10)
26	PC3 (ADC3/PCINT11)
27	PC4 (ADC4/SDA/PCINT12)
28	PC5 (ADC5/SCL/PCINT13)

No confundir el número de pin de Arduino con el número de terminal en el chip



Registros de los Puertos I/O

¿Donde están los registros I/O del puerto B ?



Hoja de datos
(data sheet)

Registros de los Puertos I/O

¿Donde están los registros I/O del puerto B ?

Capítulo 14
I/O ports



ATmega48A/PA/88A/PA/168A/PA/328/P

14.4 Register Description

14.4.1 MCUCR – MCU Control Register

Bit	7	6	5	4	3	2	1	0	
0x35 (0x55)	—	BODS ⁽¹⁾	BODSE ⁽¹⁾	PUD	—	—	IVSEL	IVCE	MCUCR
Read/Write	R	RW	RW	RW	R	—	RW	RW	
Initial Value	0	0	0	0	0	0	0	0	

Notes: 1. BODS and BODSE only available for picoPower devices ATmega48PA/88PA/168PA/328P

• Bit 4 – PUD: Pull-up Disable

When this bit is written to one, the pull-ups in the I/O ports are disabled even if the DDxn and PORTxn Registers are configured to enable the pull-ups (DDxn, PORTxn = 0b01). See "Configuring the Pin" on page 85 for more details about this feature.

14.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	RW	RW	RW	RW	RW	RW	RW	RW	
Initial Value	0	0	0	0	0	0	0	0	

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	RW	RW	RW	RW	RW	RW	RW	RW	
Initial Value	0	0	0	0	0	0	0	0	

14.4.4 PINB – The Port B Input Pins Address⁽¹⁾

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	RW	RW	RW	RW	RW	RW	RW	RW	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

14.4.5 PORTC – The Port C Data Register

Bit	7	6	5	4	3	2	1	0	
0x08 (0x28)	—	PORTC6	PORTC5	PORTC4	PORTC3	PORTC2	PORTC1	PORTC0	PORTC
Read/Write	R	RW	RW	RW	RW	RW	RW	RW	
Initial Value	0	0	0	0	0	0	0	0	

14.4.6 DDRC – The Port C Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x07 (0x27)	—	DDC6	DDC5	DDC4	DDC3	DDC2	DDC1	DDC0	DDRC
Read/Write	R	RW	RW	RW	RW	RW	RW	RW	
Initial Value	0	0	0	0	0	0	0	0	

Registros de los Puertos I/O

¿Donde están los registros I/O del puerto B ?

descripción



14.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

14.4.4 PINB – The Port B Input Pins Address⁽¹⁾

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

Registros de los Puertos I/O

- Como se configura la dirección

DDxn	PORTxn	I/O	Pull-up	Comment
0	0	Input	No	Tri-state (Hi-Z)
0	1	Input	Yes	Pxn will source current if ext. pulled low.
1	0	Output	No	Output Low (Sink)
1	1	Output	No	Output High (Source)

Veremos en otra clase la configuración de entrada con pull up y la estructura interna de los puertos I/O

Registros de los Puertos I/O

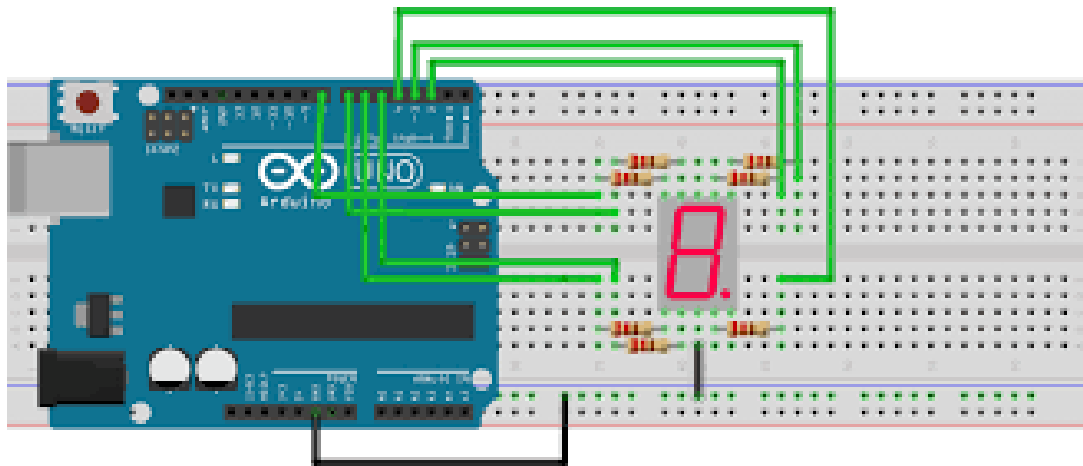
Mostrar un “1” en un display 7-segmentos

DDRD:

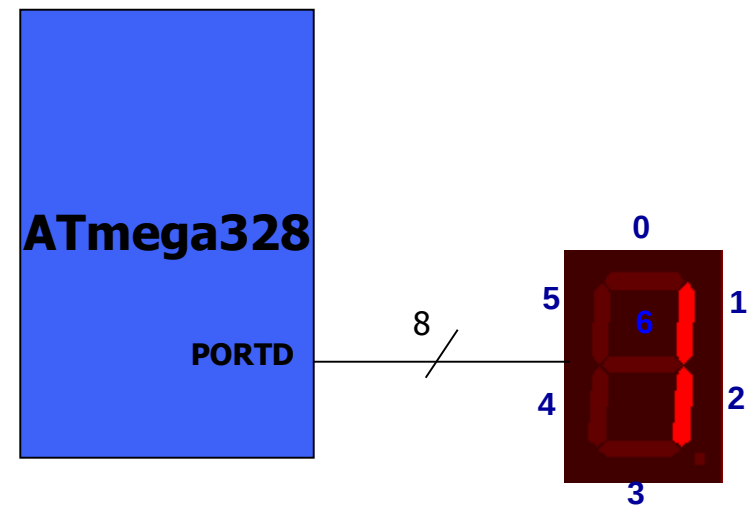
1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

PORTD:

0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---



fritzing



PORTx	DDRx	0	1
		high impedance	Out 0
	1	pull-up	Out 1

Registros de los Puertos I/O

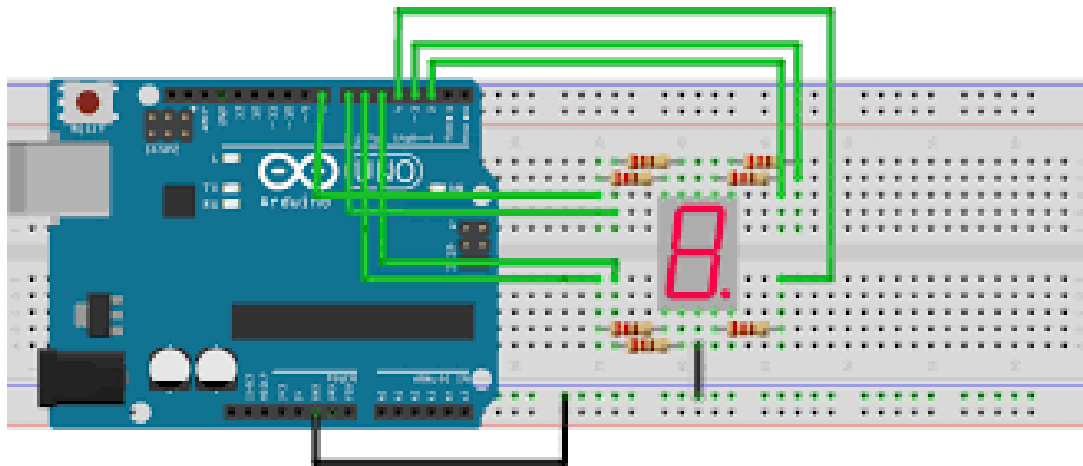
Mostrar un “3” en un display 7-segmentos.

DDRD:

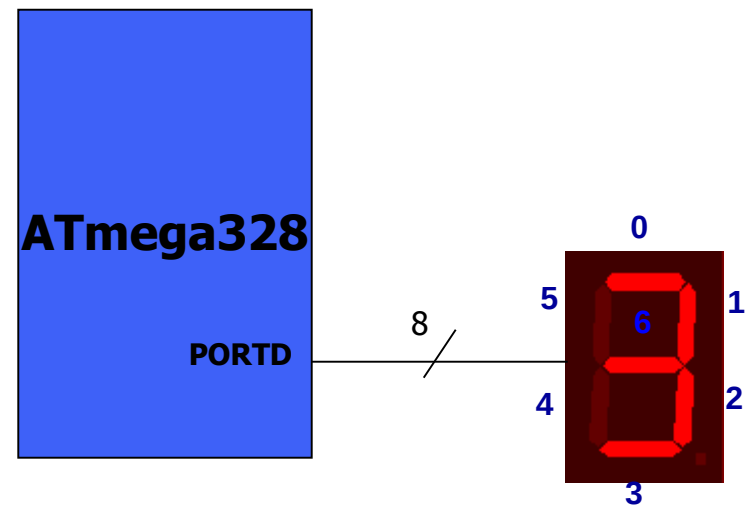
1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

PORTD:

0	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---



fritzing

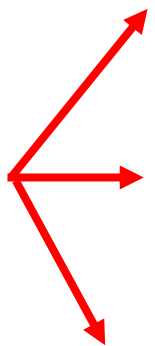


PORTx	DDRx	0	1
0		high impedance	Out 0
1		pull-up	Out 1

Programando PORTS en C

¿Cómo se utilizan los registros en C?

variables uint8 que llevan
los mismos nombres que
los registros



```
DDRB= 0b01010110;
```

```
PORTB= 0xC3;
```

```
mi_var = PINB; //leer el registro de entrada y  
copiarlo a una variable
```

La asociación entre las variables y los registros se realiza en el
archivo de cabecera `io.h`

Programando PORTS en C

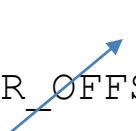
Uso de registros I/O en AVR - GCC

- Por ejemplo: Declaración del DDRB ([io328p.h](#))

```
#define DDRB __SFR_IO8(0x04)
```

- Macros para los SFR -Special Function Register ([sfr_defs.h](#)):

```
■#define __SFR_IO8(io_addr) __MMIO_uint8_t(io_addr + __SFR_OFFSET)
```

 0 I/O instr.

- Luego :

```
■#define __MMIO_uint8_t(mem_addr) (*(volatile uint8_t *) (mem_addr))
```

- Por lo tanto en el espacio I/O:

```
DDRB es *( (volatile uint8_t *) 0x04 )
```

Programando PORTS en C

Ejemplo: escribir un valor 0xAA al PORTD

```
#include <avr/io.h>

int main ()
{
    DDRD = 0xFF;           DDRD=11111111
    PORTD = 0xAA;          PORTD=10101010

    while (1);             Loop
    return 0;
}
```


Programando PORTS en C

Ejemplo: Leer, modificar y escribir un PORT

```
#include <avr/io.h>
```

← Incluye cabecera

```
int main ()
```

← Función principal

```
{
```

```
    DDRB = 0xFF;
```

← Inicialización

```
    while (1){
```

← Bucle infinito

```
        PORTD = PORTD+1;
```

← Operaciones: Leer-sumar-escribir

```
    }
```

```
    return 0;
```

← Fin función principal
(nunca se alcanza)

```
}
```

Programando PORTS en C

Ejemplo: transferir entradas a salidas

```
#include <avr/io.h>

int main ()
{
    DDRB = 0x00;
    DDRC = 0xFF;

    while (1){
        PORTC = PINB;
    }
    return 0;
}
```

Programando PORTS en C

Ejemplo: obtener PINB + PINC y escribir el resultado en PORTD.

```
#include <avr/io.h>
```

```
int main ()
```

```
{
```

```
    DDRB = 0x00;
```

Todos los bits del PORTB como entradas

```
    DDRC = 0x00;
```

Todos los bits del PORTC como entradas

```
    DDRD = 0xFF;
```

Todos los bits del PORTD como salidas

```
    while (1)
```

```
        PORTD = PINB + PINC;
```

```
        return 0;
```

```
}
```

Programando PORTS en C

Ejemplo: visualizar un contador de 8 bits en un PORT

```
#include <avr/io.h>

int main ()
{
    unsigned char z;
    DDRB = 0xFF;
    for(z=0; z<256; z++){
        PORTB=z;
    }
    while (1);
    return 0;
}
```

Resultado:

PORTB = 0x00, 0x01, 0x02, ... , 0xFF

Programando PORTS en C

Ejemplo: escribir caracteres ASCII en el PORTB

```
#include <avr/io.h>
int main ()
{
    unsigned char myList[]="012345ABCD";
    unsigned char z;
    DDRB = 0xFF;
    for(z=0;z<10;z++){
        PORTB=myList[z];
    }
    while (1);
    return 0;
}
```

¿qué representan los bits en el PORTB?

Programando PORTS en C

Ejemplo: Trabajando con números con signo (ca2)

```
#include <avr/io.h>

int main ()
{
    char num[]={-4,-3,-2,-1,0,1,2,3,4};
    unsigned char z;
    DDRB = 0xFF;
    for(z=0;z<9;z++){
        PORTB=num[z];
    }
    while (1);
    return 0;
}
```

Resultado:

PORTB = 0xFC, 0xFD, 0xFE, 0xFF, 0x00, 0x01, 0x02, 0x03, 0x04

Programando PORTS en C

Ejemplo: Conversión Binario-Decimal

```
#include <avr/io.h>
int main ()
{
    unsigned char x, binbyte, d1, d2, d3;
    DDRB = DDRC = DDRD = 0xFF;

    binbyte=0xFD;
    x=binbyte/10;
    d1=binbyte%10;
    d2=x%10;
    d3=x/10;
    PORTB=d1;
    PORTC=d2;
    PORTD=d3;
    while (1);
    return 0;
}
```

← dato inicial

0xFD = 253

← Unidad

Unidad = 3 = 0b00000011

← decena

Decena = 5 = 0b00000101

← centena

Centena = 2 = 0b00000010

Programando PORTS en C

¿Cómo accedemos a un solo terminal I/O del puerto B ?

¿Cómo modificamos 1bit sin alterar el resto del contenido?

14.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

14.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Terminal
PB0

14.4.4 PINB – The Port B Input Pins Address⁽¹⁾

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

Operadores lógicos bit a bit

Table 7-3: Bit-wise Logic Operators for C

		AND	OR	EX-OR	Inverter
A	B	A&B	A B	A^B	Y=~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	

AND

```
1110 1111
& 0000 0001
-----
0000 0001
```

OR

```
1110 1111
| 0000 0001
-----
1110 1111
```

NOT

```
~ 1110 1011
-----
0001 0100
```

Operadores de desplazamiento

data >> number of bits to be shifted right

data << number of bits to be shifted left

1110 0000 >> 3

0001 1100

0000 0001 <<2

0000 0100

Operaciones Lógicas con bits

Ejemplo: Trabajando con operadores lógicos a nivel de bits

```
#include <avr/io.h>

int main ()
{
    DDRB = 0xFF;

    while (1){

        PORTB = PORTB | 0b00010000;
        PORTB = PORTB & 0b11101111;
    }
    return 0;
}
```

Operaciones Lógicas con bits

Ejemplo: Trabajando con operadores lógicos a nivel de bits

```
#include <avr/io.h>
```

```
int main ()
```

```
{
```

```
    DDRB =0xFF;
```

```
    DDRC = 0x00;
```

```
    while (1){
```

```
        if(PINC & 0b00100000){
```

```
            PORTB=0x55;
```

```
        }
```

```
        else{
```

```
            PORTB=0xAA;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
PINC  XXXX XXXX  
& 0010 0000
```

```
00x0 0000
```

¿resultados
posibles?

Simplificaciones Lógicas con I/O

Establecer un 1 lógico en el bit 4 del PORTB

	XXXX	XXXX
	0001	0000

	xxx1	xxxx

Otra forma de escribir la mascara:

	XXXX	XXXX
	1 << 4	

	xxx1	xxxx

```
PORTB |= (1<<4); //set bit 4 (5th bit) of PORTB
```

Notar que de esta manera no se modifican los otros bits del mismo registro

Simplificaciones Lógicas con I/O

Establecer un 0 lógico en el bit 4 del PORTB

```
      XXXX  XXXX
&    1110 1111
-----
      xxx0  xxxx
```

Otra forma de escribir la mascara:

```
      XXXX  XXXX
&    ~(1 << 4)
-----
      xxx0  xxxx
```

```
PORTB &= ~(1<<4); //clear bit 4 (5th bit) of PORTB
```

Simplificaciones Lógicas con I/O

¿está el bit 5 de PINC en 1?

PINC XXXX XXXX
& 0010 0000 == (1 << 5)

 00x0 0000
 x puede ser 0 ó 1

```
if( ((PINC & (1<<5)) != 0)    //check bit 5 (6th bit)
```

o directamente:

```
if(((PINC & (1<<5)))    //check bit 5 (6th bit)
```

Operaciones sobre registros I/O

- Resumiendo

```
X |= (1 << Bitnumber);    // setear un bit de la variable x, SIN AFECTAR EL RESTO
X &= ~(1 << Bitnumber);   // borrar un bit de la variable x, SIN AFECTAR EL RESTO
```

- Ejemplo:

```
#include <avr/io.h>
...
PORTD |= (1 << 2);        //setear bit 2 del port D
PORTD &= ~(1 << 2);       //borrar bit 2 del port D
```

- Utilizando los macros definidos en <avr/io.h>

```
#include <avr/io.h>
...
DDRD &= ~( (1<<PORTD0) | (1<<PORTD3) );    /* Setear PD0 y PD3 como inputs */
PORTD |= (1<<PORTD0) | (1<<PORTD3);        /* Habilitar sus Pull UP s internos */
```


Operaciones sobre registros I/O

- Implementación y Optimización del compilador:

```
PORTD |= (1<<PORTD0);           //set bit 0 del port A
PORTD |= (1<<PORTD2) | (1<<PORTD3) | (1<<PORTD4); //set bits 2, 3 y 4 del port A
```

- En Assembler queda:

```
PORTD |= (1<<PORTD0);
6c: d8 9a    sbi      0x1b, 0
```

```
PORTD |= (1<<PORTD2) | (1<<PORTD3) | (1<<PORTD4);
6e: 8b b3    in       r24, PORTD
70: 8c 61    ori      r24, 0x1C
72: 8b bb    out      PORTD, r24
```

Utilizando AVR LIBC

- Un retardo de tiempo puede implementarse simplemente así:

...pero es dependiente del compilador y las optimizaciones y es difícil de ajustar

```
void delay(void)
{
    unsigned short i;
    for(i = 0; i < 42150; i++)
    { ; }
}
```

- Biblioteca de retardos:
 - Se debe especificar ANTES la constante F_CPU con la frecuencia de reloj [MHz]

```
#define F_CPU 16000000UL
#include <util/delay.h>
```

```
    _delay_us(200);    //200 microseconds
    _delay_ms(100);    //100 milliseconds
```

- Se implementa directamente en assembler

Utilizando AVR LIBC

- Bibliotecas de funciones matemáticas (math.h)

Function	Avr2	Avr4 (Hardware MUL)
__addsf3 (1.234, 5.678)	113	108
__mulsf3 (1.234, 5.678)	375	138
__divsf3 (1.234, 5.678)	466	465
acos (0.54321)	4411	2455
asin (0.54321)	4517	2556
atan (0.54321)	4710	2271
atan2 (1.234, 5.678)	5270	2857
cbrt (1.2345)	2684	2555
ceil (1.2345)	177	177
cos (1.2345)	3387	1671
cosh (1.2345)	4922	2979
exp (1.2345)	4708	2765
fdim (5.678, 1.234)	111	111
floor (1.2345)	180	180
fmax (1.234, 5.678)	39	37
fmin (1.234, 5.678)	35	35
fmod (5.678, 1.234)	131	131
frexp (1.2345, 0)	42	41
hypot (1.234, 5.678)	1341	866
ldexp (1.2345, 6)	42	42
log (1.2345)	4142	2134
log10 (1.2345)	4498	2260
modf (1.2345, 0)	433	429
pow (1.234, 5.678)	9293	5047
round (1.2345)	150	150
sin (1.2345)	3353	1653
sinh (1.2345)	4946	3003
sqrt (1.2345)	494	492
tan (1.2345)	4381	2426
tanh (1.2345)	5126	3173
trunc (1.2345)	178	178

Acceso a la memoria de programa

```
#include <avr/pgmspace.h>

const unsigned char PROGMEM lookup[] = {5,6,7,4};

int main(void)
{
    unsigned char a;
    a = pgm_read_byte(&lookup[i]);

    while (1);
}
```

En *pgmspace.h* también se encuentran listadas las funciones:

pgm_read_word
pgm_read_lword
pgm_read_float

Acceso a la memoria EEPROM

```
#include <avr/io.h>
#include <avr/eeprom.h>

unsigned char EEMEM myVar; //reserve a location in EEPROM

int main(void)
{
    DDRC = 0xFF;

    if((PINB&(1<<0)) != 0) //if PB0 is HIGH
        eeprom_write_byte(&myVar, 'G'); //read from EEPROM
    else
        PORTC = eeprom_read_byte(&myVar); //write to EEPROM

    while (1);
}
```

En *eeprom.h* también se encuentran listadas las funciones:

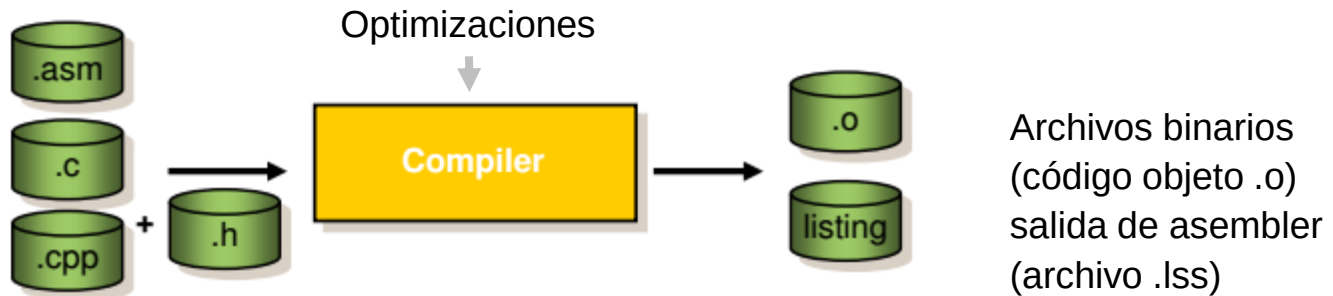
<code>eeprom_write_word</code>	<code>eeprom_read_word</code>
<code>eeprom_write_lword</code>	<code>eeprom_read_lword</code>
<code>eeprom_write_float</code>	<code>eeprom_read_float</code>

Conociendo las Herramientas

- El compilador y el Linker (el proceso Build)

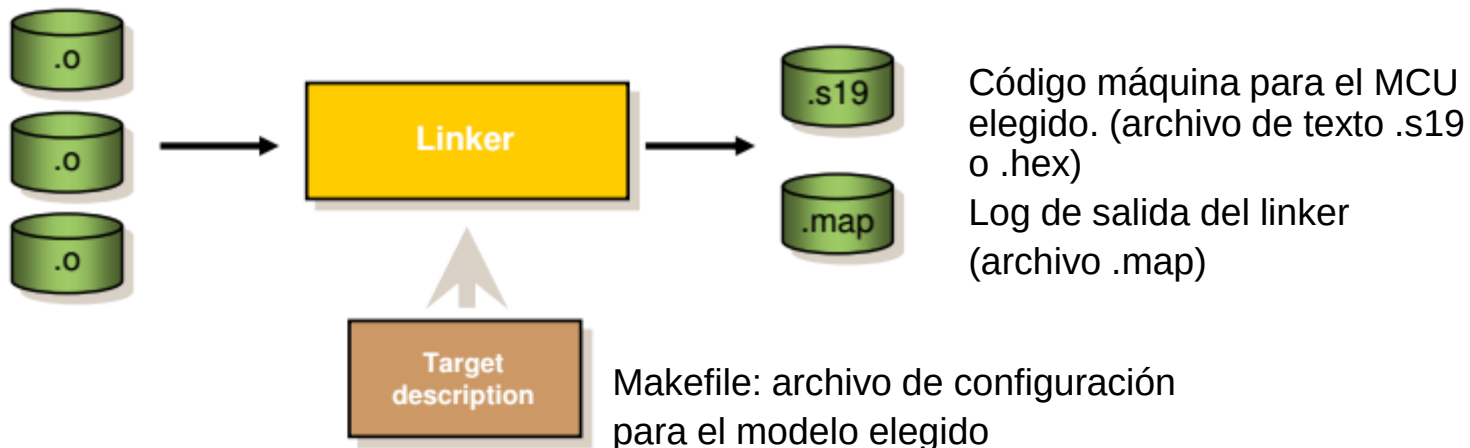
Paso 1

Archivos fuentes del proyecto



Paso 2

archivos objetos del proyecto + bibliotecas precompiladas



Conociendo las Herramientas

- Una vez que hacemos “BUILD” podemos hacernos las siguientes preguntas:

¿cómo sé si mi código fue optimizado?

=>el `archivo.lst` (listing file)

contiene el código assembler generado por compilador

¿Cómo se distingue el binario de un microcontrolador a otro?

=>el archivo del proyecto `makefile` (lista de comandos)

invoca al compilador y al linker pasándole la información del target (MCU) y el modelo de memoria utilizado.

¿En qué dirección de memoria está mi código? ¿y mis variables?

=> el `archivo.map`

!!! Es un log del resultado del trabajo del linker

Conociendo las Herramientas

- ¿Qué es un código de startup?

Es el código que prepara (inicializa) el MCU para poder ejecutar la aplicación del usuario.

Generalmente está escrito en assembler por el diseñador de la herramienta.

Las funciones principales son:

- 1 -Deshabilitar las interrupciones
- 2 –Inicializar variables globales o estáticas con valor !=0 (copia de ROM a RAM)
- 3 - Inicializar variables globales o estáticas no inicializadas en cero (ANSI-C)
- 4 -Reservar espacio e inicializar el puntero de pila (Stack Pointer)
- 5 -Crear e inicializar el Heap (Dynamic Memory Allocation avr-libc)
- 6 -Ejecutar main()

En el caso del AVR vemos el código startup insertado en el archivo .iss